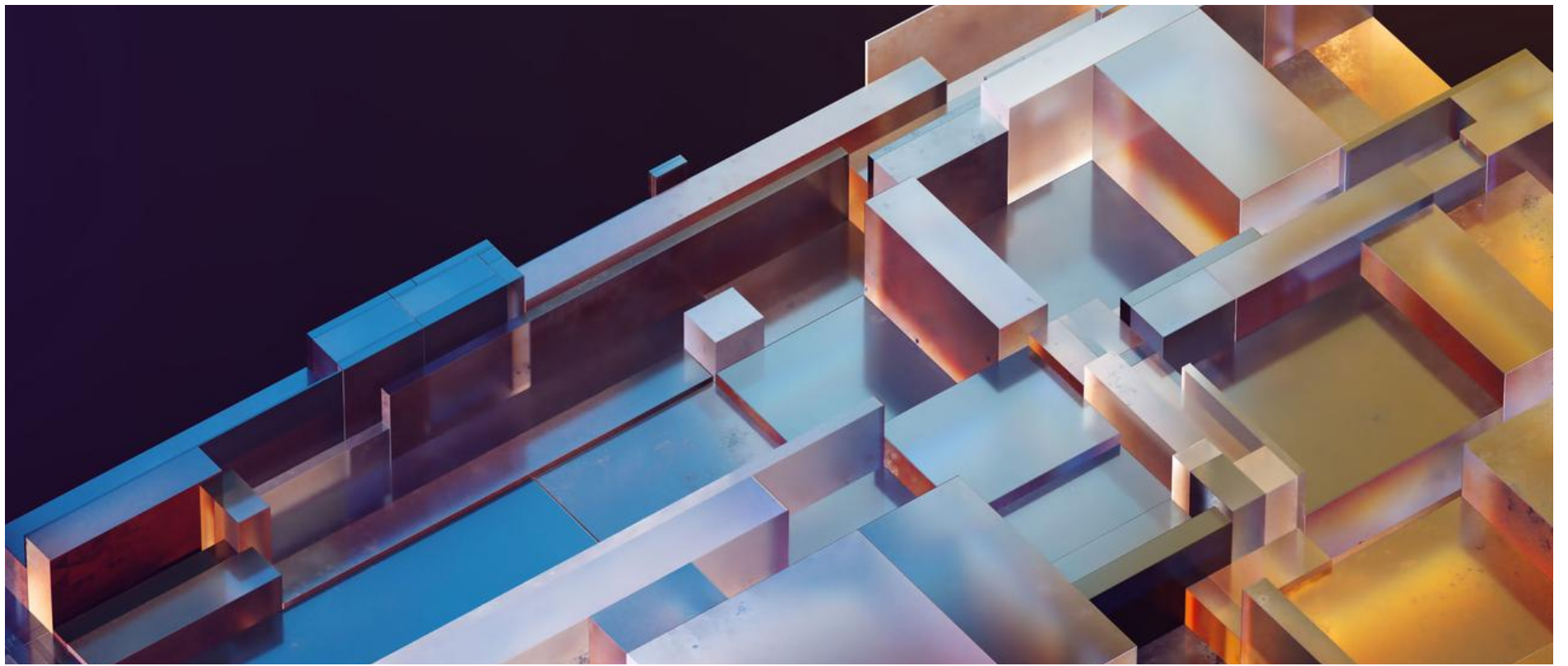




Vite All in One: Vite工程化开发实践与构建原理解析

1. 课程代码

 encode-chatgpt-vue.zip
370.11KB



2. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)

3. Vite 最佳实践

📖 12. AI智能助手

vue3最佳实践：基于 vue3、naiveUI、vite4、vue-router4、pinia2实现了AI智能会话系统，支持包含：

1. 基于 npm chatgpt实现双模型切换，支持openai key 及 accessToken接入；

2. 基于stream及会话parentId 实现会话上下文逻辑封装；
3. 基于pinia hooks 实现不同状态逻辑的封装，提供访问权限控制；
4. 基于 i18n、naive ui 实现 国际化及主题切换；
5. 基于 axios interceptor 实现会话 token的校验及最大调用次数的限制；
6. 基于 env 抽离通用配置，支持环境快速搭建；

4. Vite 生态 & 性能优化

| <https://github.com/vitejs/awesome-vite>

<https://github.com/pure-admin/vue-pure-admin>

4.1 常见的性能优化分析方式

用于生产部署的负载性能分析：

- [PageSpeed Insights](#)
- [WebPageTest](#)

用于本地开发期间的性能分析：

- [Chrome 开发者工具 “性能” 面板](#)
 - `app.config.performance` 将会开启 Vue 特有的性能标记，标记在 Chrome 开发者工具的性能时间线上。
- [Vue 开发者扩展](#)也提供了性能分析的功能。

4.2 页面加载优化

4.2.1 选用正确的架构

如果你的用例对页面加载性能很敏感，请避免将其部署为纯客户端的 SPA，而是让服务器直接发送包含用户想要查看的内容的 HTML 代码。纯客户端渲染存在首屏加载缓慢的问题，这可以通过[服务器端渲染 \(SSR\)](#) 或[静态站点生成 \(SSG\)](#) 来缓解。如果应用对交互性要求不高，你还可以使用传统的后端服务器来渲染 HTML，并在客户端使用 Vue 对其进行增强。

如果你的主应用必须是 SPA，但还有其他的营销相关页面 (落地页、关于页、博客等)，请单独部署这些页面！理想情况下，营销页面应该是包含尽可能少 JS 的静态 HTML，并用 SSG 方式部署。

4.2.2 包体积与 Tree-shaking 优化

一个最有效的提升页面加载速度的方法就是压缩 JavaScript 打包产物的体积。当使用 Vue 时有下面一些办法来减小打包产物体积：

- 尽可能地采用构建步骤
 - 如果使用的是相对现代的打包工具，许多 Vue 的 API 都是可以被 [tree-shake](#) 的。举例来说，如果你根本没有使用到内置的 `<Transition>` 组件，它将不会被打包进入最终的产物里。Tree-shaking 也可以移除你源代码中其他未使用到的模块。
 - 当使用了构建步骤时，模板会被预编译，因此我们无须在浏览器中载入 Vue 编译器。这在同样最小化加上 gzip 优化下会相对缩小 14kb 并避免运行时的编译开销。
- 在引入新的依赖项时要小心包体积膨胀！在现实的应用中，包体积膨胀通常因为无意识地引入了过重的依赖导致的。
 - 如果使用了构建步骤，应当尽量选择提供 ES 模块格式的依赖，它们对 tree-shaking 更友好。举例来说，选择 `lodash-es` 比 `lodash` 更好。
 - 查看依赖的体积，并评估与它所提供的功能之间的性价比。如果依赖对 tree-shaking 友好，实际增加的体积大小将取决于你从它之中导入的 API。像 [bundlejs.com](#) 这样的工具可以用来做快速的检查，但是根据实际的构建设置来评估总是最准确的。

4.2.3 代码分割

代码分割是指构建工具将构建后的 JavaScript 包拆分为多个较小的，可以按需或并行加载的文件。通过适当的代码分割，页面加载时需要的功能可以立即下载，而额外的块只在需要时才加载，从而提高性能。

像 Rollup (Vite 就是基于它之上开发的) 或者 webpack 这样的打包工具可以通过分析 ESM 动态导入的语法来自动进行代码分割：

Code block

```
1  // lazy.js 及其依赖会被拆分到一个单独的文件中
2  // 并只在 `loadLazy()` 调用时才加载
3  function loadLazy() {
4    return import('./lazy.js')
5  }
```

懒加载对于页面初次加载时的优化帮助极大，它帮助应用暂时略过了那些不是立即需要的功能。在 Vue 应用中，这可以与 Vue 的[异步组件](#)搭配使用，为组件树创建分离的代码块：

Code block

```
1  import { defineAsyncComponent } from 'vue'
2
3  // 会为 Foo.vue 及其依赖创建单独的一个块
4  // 它只会按需加载
5  // （即该异步组件在页面中被渲染时）
6  const Foo = defineAsyncComponent(() => import('./Foo.vue'))
```

4.3 更新优化

4.3.1 Props 稳定性

在 Vue 之中，一个子组件只会在其至少一个 props 改变时才会更新。思考以下示例：

Code block

```
1  <ListItem
2    v-for="item in list"
3    :id="item.id"
4    :active-id="activeId" />
```

在 `<ListItem>` 组件中，它使用了 `id` 和 `activeId` 两个 props 来确定它是否是当前活跃的那一项。虽然这是可行的，但问题是每当 `activeId` 更新时，列表中的每一个 `<ListItem>` 都会跟着更新！

理想情况下，只有活跃状态发生改变的项才应该更新。我们可以将活跃状态比对的逻辑移入父组件来实现这一点，然后让 `<ListItem>` 改为接收一个 `active` prop：

Code block

```
1  <ListItem
2    v-for="item in list"
3    :id="item.id"
4    :active="item.id === activeId" />
```

4.3.2 v-once

`v-once` 是一个内置的指令，可以用来渲染依赖运行时数据但无需再更新的内容。它的整个子树都会在未来的更新中被跳过。

在随后的重新渲染，元素/组件及其所有子项将被当作静态内容并跳过渲染。这可以用来优化更新时的性能。

Code block

```
1  <!-- 单个元素 -->
2  <span v-once>This will never change: {{msg}}</span>
3  <!-- 带有子元素的元素 -->
4  <div v-once>
5    <h1>comment</h1>
```

```
6     <p>{{msg}}</p>
7   </div>
8   <!-- 组件 -->
9   <MyComponent v-once :comment="msg" />
10  <!-- `v-for` 指令 -->
11  <ul>
12    <li v-for="i in list" v-once>{{i}}</li>
13  </ul>
```

4.3.3 v-memo

`v-memo` 是一个内置指令，可以用来有条件地跳过某些大型子树或者 `v-for` 列表的更新。

缓存一个模板的子树。在元素和组件上都可以使用。为了实现缓存，该指令需要传入一个固定长度的依赖值数组进行比较。如果数组里的每个值都与最后一次的渲染相同，那么整个子树的更新将被跳过。举例来说：

template

Code block

```
1   <div v-memo="[valueA, valueB]">
2     ...
3   </div>
```

当组件重新渲染，如果 `valueA` 和 `valueB` 都保持不变，这个 `<div>` 及其子项的所有更新都将被跳过。实际上，甚至虚拟 DOM 的 vnode 创建也将被跳过，因为缓存的子树副本可以被重新使用。

正确指定缓存数组很重要，否则应该生效的更新可能被跳过。`v-memo` 传入空依赖数组 (`v-memo="[]"`) 将与 `v-once` 效果相同。

与 `v-for` 一起使用

`v-memo` 仅用于性能至上场景中的微小优化，应该很少需要。最常见的情况可能是有助于渲染海量 `v-for` 列表 (长度超过 1000 的情况)：

template

Code block

```
1   <div v-for="item in list" :key="item.id" v-memo="[item.id === selected]">
2     <p>ID: {{ item.id }} - selected: {{ item.id === selected }}</p>
3     <p>...more child nodes</p>
4   </div>
```

当组件的 `selected` 状态改变，默认会重新创建大量的 vnode，尽管绝大部分都跟之前是一模一样的。`v-memo` 用在这里本质上是在说“只有当该项的被选中状态改变时才需要更新”。这使得每个选中状态没有变的项能完全重用之前的 vnode 并跳过差异比较。注意这里 memo 依赖数组中并不需要包含 `item.id`，因为 Vue 也会根据 item 的 `:key` 进行判断。

4.4 通用优化

4.4.1 虚拟滚动

根据可视区的高度以及items中每一项的高度(itemSize，可为高度或者是横向滑动的宽度)来决定页面展示多少个item，能显示的item包装后放到了pool数组中进行渲染，页面滚动的时候动态的修改pool数组。为了在滚动的时候尽可能的减少开销，pool中超出范围的view会回收到复用池，pool中新增的view会优先从复用池中取出view，如果没有复用的才会新增。

4.4.2 避免不必要的组件抽象

有些时候我们会去创建[无渲染组件](#)或高阶组件 (用来渲染具有额外 props 的其他组件) 来实现更好的抽象或代码组织。虽然这并没有什么问题，但请记住，组件实例比普通 DOM 节点要昂贵得多，而且为了逻辑抽象创建太多组件实例将会导致性能损失。

需要提醒的是，只减少几个组件实例对于性能不会有明显的改善，所以如果一个用于抽象的组件在应用中只会渲染几次，就不用操心去优化它了。考虑这种优化的最佳场景还是在大型列表中。想象一下一个有 100 项的列表，每项的组件都包含许多子组件。在这里去掉一个不必要的组件抽象，可能会减少数百个组件实例的无谓性能消耗。

5. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



6. Vite 插件化开发

<https://cn.vite.dev/guide/api-plugin>

6.1 打包性能优化

1. html处理

<https://github.com/vbenjs/vite-plugin-html>

vite-plugin-html 是一个 Vite 插件，用于将传统 HTML 文件作为输出文件使用。该插件基于 html-webpack-plugin 插件创建，但是是专门为 Vite 构建的优化工具，不依赖于 webpack。

安装：

Code block

```
1  pnpm add vite-plugin-html -D
```

使用：

Code block

```
1  import { defineConfig, Plugin } from 'vite'
2  import vue from '@vitejs/plugin-vue'
3
4  import { createHtmlPlugin } from 'vite-plugin-html'
5
6  export default defineConfig({
7    plugins: [
8      vue(),
9      createHtmlPlugin({
10        minify: true,
11        /**
12         * 在这里写entry后，你将不需要在`index.html`内添加 script 标签，原有标签需要删除
13         * @default src/main.ts
14         */
15        entry: 'src/main.ts',
16        /**
17         * 如果你想将 `index.html` 存放在指定文件夹，可以修改它，否则不需要配置
18         * @default index.html
19         */
20        template: 'public/index.html',
21        /**
22         * 需要注入 index.html ejs 模版的数据
23         */
24        inject: {
25          data: {
26            title: 'index',
27            injectScript: `<script src="./inject.js"></script>`,
28          },
29          tags: [
30            {
31              injectTo: 'body-prepend',
32              tag: 'div',
33              attrs: {
34                id: 'tag',
35              },
36            },
37          ],
38        ],
39      },
40    ],
41  })
```

```
40     }},  
41   ],  
42 })
```

2. 图片压缩

<https://github.com/FatehAK/vite-plugin-image-optimizer>

vite-plugin-image-optimizer 是一个 Vite 插件，用于自动优化应用程序中的图像文件。该插件可以帮助开发者减少应用程序的加载时间和带宽使用，从而提高用户体验和网站性能。

安装：

Code block

```
1  pnpm add vite-plugin-image-optimizer -D
```

使用：

Code block

```
1  import { ViteImageOptimizer } from 'vite-plugin-image-optimizer';  
2  import { defineConfig } from 'vite';  
3  
4  export default defineConfig(() => {  
5    return {  
6      plugins: [  
7        ViteImageOptimizer({  
8          /* pass your config */  
9        }),  
10     ],  
11   };  
12 });
```

3. gzip代码压缩

<https://github.com/vbenjs/vite-plugin-compression>

vite-plugin-compression是一个基于Vite的插件,用于gzip或Brotli压缩你的资源，从而减少页面的加载时间和网络带宽，提高用户访问速度和体验。

安装：

Code block

```
1  pnpm add vite-plugin-compression -D
```

使用：

Code block

```
1  import viteCompression from 'vite-plugin-compression';  
2  export default () => {  
3    return {  
4      plugins: [viteCompression()],  
5    };  
6  };
```

4. 生产环境移除console

| <https://github.com/xiaoxian521/vite-plugin-remove-console>

vite-plugin-remove-console是一款生产环境删除所有指定console类型的vite插件。

安装：

Code block

```
1  pnpm add vite-plugin-remove-console -D
```

使用：

Code block

```
1  import { defineConfig } from "vite";
2  import vue from "@vitejs/plugin-vue";
3  import removeConsole from "vite-plugin-remove-console";
4  // https://vitejs.dev/config/
5  export default defineConfig({
6    plugins: [vue(), removeConsole()]
7  });
```

5. 依赖分析

| <https://github.com/btd/rollup-plugin-visualizer>

rollup-plugin-visualizer 是一个用于可视化 Rollup 打包输出文件的插件，它可以帮助开发者分析和优化代码依赖关系，从而提高代码性能和可读性。

安装：

Code block

```
1  pnpm add rollup-plugin-visualizer -D
```

使用：

Code block

```
1  // vite.config.ts
2  import vue from "@vitejs/plugin-vue";
3  import { defineConfig } from 'vite'
4  import type { PluginOption } from 'vite';
5  import visualizer from 'rollup-plugin-visualizer';
6  export default defineConfig({
7    plugins: [
8      vue(),
9      visualizer({
10        // template: 'treemap', // sunburst | treemap | network | raw-data | list
11        filename: './node_modules/.cache/visualizer/stats.html',
12        open: true,
13        gzipSize: true,
14        brotliSize: true,
15      }) as PluginOption
16    ]
17  });
```

6.2 开发体验优化

1. 自动导入第三方ui库组件

<https://github.com/unplugin/unplugin-vue-components>

unplugin-vue-components 是一个 Vite 插件，用于自动化地将您的 Vue.js 组件库导出为独立包。该插件可以帮助您更轻松地编写和共享 Vue 组件，并提供更好的开发体验和代码复用性。

安装：

Code block

```
1 pnpm add unplugin-vue-components -D
```

使用：

Code block

```
1 import vue from "@vitejs/plugin-vue";
2 import { defineConfig } from 'vite';
3 import Components from 'unplugin-vue-components/vite';
4 import { AntDesignVueResolver } from 'unplugin-vue-components/resolvers';
5 export default defineConfig({
6   plugins: [
7     vue(),
8     Components({
9       resolvers: [
10        AntDesignVueResolver({
11          importStyle: 'less',
12          resolveIcons: true,
13        }),
14      ],
15      dts: 'types/components.d.ts',
16      include: [/\.vue
17 $/, /\.vue\?vue/, /\.md$/],
18    /],
19    version: 3,
20  })
21  ]
22  })
```

2. 自动引入类型定义

<https://github.com/xiaoxian521/vite-plugin-remove-console>

unplugin-auto-import 是一个 Vite 插件，可以自动将需要的模块或库注入到 JavaScript 或 TypeScript 文件中。该插件可以帮助开发者减少手动导入模块的工作量，并且可以防止由于拼写错误或路径错误导致的编译错误。

安装：

Code block

```
1 pnpm add unplugin-auto-import -D
```

使用：

Code block

```
1 import vue from "@vitejs/plugin-vue";
2 import { defineConfig } from 'vite';
3 import type { PluginOption } from 'vite';
4 import AutoImport from 'unplugin-auto-import/vite';
5 export default defineConfig({
6   plugins: [
```

```

7      vue(),
8      AutoImport({
9          imports: [
10             'vue',
11             'vue-router',
12             {
13                 '@vueuse/core': [
14                     // named imports
15                     'useMouse', // import { useMouse } from '@vueuse/core',
16                     'usePreferredDark',
17                     'useDark',
18                     'useTitle',
19                     // alias
20                     ['useFetch', 'useMyFetch'], // import { useFetch as useMyFetch } from
 '@vueuse/core',
21                 ],
22                 axios: [
23                     // default imports
24                     ['default', 'axios'], // import { default as axios } from 'axios',
25                 ],
26             },
27         ],
28         // resolvers: [AntDesignVueResolver()],
29         vueTemplate: true,
30         cache: true,
31         dirs: ['src/hooks', 'src/components'],
32         dts: 'types/auto-imports.d.ts',
33     })
34 ]
35 })

```

3. svg作为组件使用

[GitHub - jpkleemans/vite-svg-loader: Vite plugin to load SVG files as Vue components](#)

vite-svg-loader 是一个 Vite 插件，用于加载和处理 SVG 文件。它可以帮助开发者在应用程序中无缝使用 SVG 图像，并提供了一些额外的功能来优化和定制 SVG 图像的使用。

安装：

Code block

```
1  pnpm add vite-svg-loader -D
```

使用：

Code block

```

1  import vue from "@vitejs/plugin-vue";
2  import { defineConfig } from 'vite';
3  import type { PluginOption } from 'vite';
4  import svgLoader from 'vite-svg-loader';
5  export default defineConfig({
6      plugins: [
7          vue(),
8          svgLoader({
9              defaultImport: 'url', // or 'raw'
10          })
11      ]
12  })

```

4. 在md文档中直接使用vue组件

<https://github.com/antfu/vite-plugin-md>

vite-plugin-md 是一个 Vite 插件，用于在 Vue 项目中加载和渲染 Markdown 文件。它可以帮助开发者更方便地在应用程序中使用和管理 Markdown 内容，将 Markdown 文件转换为可交互的 HTML 页面。

安装：

Code block

```
1 pnpm add vite-plugin-md -D
```

使用：

Code block

```
1 import vue from "@vitejs/plugin-vue";
2 import { defineConfig } from 'vite';
3 import Markdown from 'vite-plugin-md';
4 export default defineConfig({
5   plugins: [
6     vue({
7       include: [/\.vue
8 $/, /\.md$/], // <--
9   }),
10   // 为了解决 Error [ERR_PACKAGE_PATH_NOT_EXPORTED], 在package.json增加 "type": "module"
11   Markdown(),
12 ],
13 })
```

5. 移动端vconsole

<https://github.com/vadxq/vite-plugin-vconsole>

一个适用于 Vite v2+ 的插件，帮助开发者在各个环境下方便使用 VConsole 的功能。可以方便配置区分环境，根据环境动态加载 VConsole，支持多页面配置。

安装：

Code block

```
1 pnpm add vite-plugin-vconsole -D
```

使用：

Code block

```
1 // tips: 如果引用path提示不存在，可以引入@types/node包
2 import { defineConfig } from 'vite';
3 import vue from '@vitejs/plugin-vue';
4 import { viteVConsole } from 'vite-plugin-vconsole';
5 import * as path from 'path'
6 // https://vitejs.dev/config/
7 export default defineConfig({
8   plugins: [
9     vue(),
10    viteVConsole({
11      entry: path.resolve('src/main.ts'), // 或者可以使用这个配置: [path.resolve('src/main.ts')]
12      enabled: true, // 可自行结合 mode 和 command 进行判断
13      config: {
```

```
14         maxLogNumber: 1000,
15         theme: 'dark'
16     }
17 })
18 ]
19 });
```

6. mock服务使用

<https://github.com/vbenjs/vite-plugin-mock>

vite 的数据模拟插件，是基于 vite.js 开发的。并同时支持本地环境和生产环境。Connect 服务中间件在本地使用，mockjs 在生产环境中使用。

安装：

Code block

```
1 pnpm add mockjs
2 pnpm add vite-plugin-mock -D
```

使用：

- 开发环境：

Code block

```
1 // vite.config.ts 配置
2 import { UserConfigExport, ConfigEnv } from 'vite'
3 import { viteMockServe } from 'vite-plugin-mock'
4 import vue from '@vitejs/plugin-vue'
5 export default ({ command }: ConfigEnv): UserConfigExport => {
6     return {
7         plugins: [
8             vue(),
9             viteMockServe({
10                 mockPath: 'mock',
11                 enable: true,
12             }),
13         ],
14     }
15 }
16 // viteMockSer 配置
17 {
18     mockPath?: string;
19     ignore?: RegExp | ((fileName: string) => boolean);
20     watchFiles?: boolean;
21     enable?: boolean;
22     ignoreFiles?: string[];
23     configPath?: string;
24     logger?: boolean;
25 }
```

7. 依赖预构建

vite在开发环境使用了esbuild，在进入新页面时，如果有引入新的依赖浏览器才会加载新依赖。为了优化开发体验，官方支持[依赖预构建](#)。

所谓的预构建是为了开发时将某些依赖预先将esm转换成单个模块。这样牺牲了一些项目启动时间但是在切换页面时更流程不会等待太长时间。

依赖预构建仅适用于开发模式，并使用 esbuild 将依赖项转换为 ES 模块。在生产构建中，将使用 @rollup/plugin-commonjs。

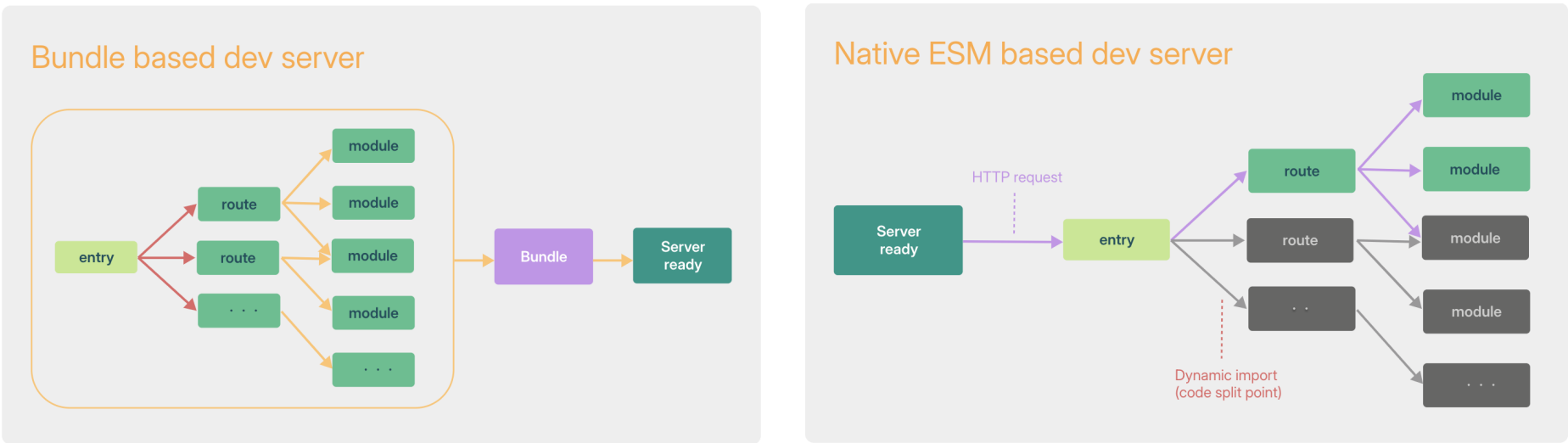
Code block

```
1 // vite.config.ts
2 import type { UserConfig, ConfigEnv } from 'vite';
3 import { defineConfig, loadEnv } from 'vite';
4 export default defineConfig(({ mode }: ConfigEnv): UserConfig => {
5   return {
6     // ...其他配置项
7     // 依赖优化 - 预构建
8     optimizeDeps: {
9       include: ['vue', 'pinia', 'vue-router', 'ant-design-vue/es', '@vueuse/core'],
10    }
11  }
12 })
```

更多配置项字段请看官方文档:[依赖优化选项](#)

7. Vite 本地构建原理

在 `Vite` 官网有两张对比图能够非常直观的对比两者的区别。



这张图代表的是基于打包器的构建方式（webpack就是其中之一），它在启动服务之前，需要从入口开始扫描整个项目的依赖关系，然后基于依赖关系构建整个应用生成bundle，最后才会启动开发服务器。这就是这类构建方式为什么慢的原因，并且整个构建时间会随着项目的变大变的越来越长！

这张图代表的是基于 `ES Module` 的构建方式（比如：Vite），这张图是不是能够很直观说明为什么 `Vite` 会非常快，因为它上来就直接启动开发服务器，然后在浏览器请求源码时进行转换并按需提供源码。根据情景动态导入代码，即只在当前页面上实际使用时才会被处理。

也就是它不需要扫描整个项目并且打包，不打包的话那它是如何让浏览器拿到分散在项目中的各个模块呢？

这一切都要得益于浏览器支持ESM的模块化方案，当浏览器识别到模块内的 `ESM` 方式导入的模块时，会自动去帮我们查找对应的内容

这就是为什么 `vite` 项目的模版文件中的 `script` 标签需要加上 `type=module`，而 `webpack` 项目不需要。

Code block

```
1 <script type="module" src="/src/main.ts"></script>
```

7.1 vite快的原因

其实上面已经能够说明 `vite` 为什么会比 `webpack` 快了，但还有另外一个点在上图中并没有表现出来。

`Vite` 会在一开始将项目中的所有模块分为**源码**和**依赖**两类

- **源码**指的是我们自己写的代码，这类代码可能需要转换（例如 JSX，CSS 或者 Vue/Svelte 组件），并且时常会被编辑。Vite 会以 **原生 ESM** 方式提供源码，同时并不是所有的源码都需要同时被加载（例如基于路由拆分的代码模块）。

- **依赖**大多为在开发时不会变动的纯 JavaScript。一些较大的依赖（例如有上百个模块的组件库）处理的代价也很高。依赖也通常会存在多种模块化格式（例如 ESM 或者 CommonJS）。Vite 将会使用 esbuild 预构建依赖。esbuild 使用 Go 编写，并且比以 JavaScript 编写的打包器预构建依赖快 10-100 倍。

总结来说就是：基于ESM模块化方案 + 预构建

7.2 使用预构建的原因

Vite 使用依赖预构建的原因主要有以下两点：

- **兼容CommonJS与UMD**：在开发阶段中，Vite 的开发服务器将所有代码视为原生 ES 模块。因此，Vite 必须先将以 CommonJS 或 UMD 形式提供的依赖项转换为 ES 模块。
- **性能**：为了提高后续页面的加载性能，Vite 将那些具有许多内部模块的 ESM 依赖项转换为单个模块。

可以来看个例子：

我们引入 `lodash-es` 工具包中的 `debounce` 方法，此时它理想状态应该是只发出一个请求

Code block

```
1 import { debounce } from 'lodash-es'
```

事实也是这样

Name	×	Headers	Payload	Preview	Response	Initiator	Time
127.0.0.1	1			// node_modules/.pnpm/lodash-es@4.17.21/node_modules/lodash-es/index.js			
client	2			var freeGlobal = typeof global == "object" && global;			
main.ts	3			var freeGlobal_default = freeGlobal;			
hook-exec.js	4						
vue.js?v=68775d13	5			// node_modules/.pnpm/lodash-es@4.17.21/node_modules/lodash-es/index.js			
lodash-es.js?v=0a59e791	6			var freeSelf = typeof self == "object" && self && self.constructor && self.constructor.prototype && self.constructor.prototype.__proto__			
App.vue	7			var root = freeGlobal_default freeSelf Function("return this")();			
env.mjs	8			var root_default = root;			
App.vue?vue&type=style&index=...	9						
__x00__plugin-vue:export-helper	10			// node_modules/.pnpm/lodash-es@4.17.21/node_modules/lodash-es/index.js			
127.0.0.1	11			var Symbol = root_default.Symbol;			
detector-exec.js	12			var Symbol_default = Symbol;			
141.327ce5c7.js	13						
vite.svg	14			// node_modules/.pnpm/lodash-es@4.17.21/node_modules/lodash-es/index.js			
	15			var objectProto = Object.prototype;			
	16			var hasOwnProperty = objectProto.hasOwnProperty;			
	17			var nativeObjectToString = objectProto.toString;			
	18			var symToStringTag = Symbol_default ? Symbol_default.prototype.toStringTag : undefined;			
	19			function getRawTag(value) {			
	20			var isOwn = hasOwnProperty.call(value, symToStringTag);			

但这是预构建的功劳，如果我们对 `lodash-es` 关闭预构建呢？

vite 配置文件加上如下代码，再来试试：

Code block

```
1 // vite.config.js
2 optimizeDeps: {
3   exclude: ['lodash-es']
4 }
```

Name	Status	Protocol	Type	Initiator
_getMapData.js?v=6ab5...	200	http/1.1	script	_mapCacheDe
_getValue.js?v=6ab58...	200	http/1.1	script	_getNative.js?
_listCacheClear.js?v=6...	200	http/1.1	script	_ListCache.js'
_listCacheDelete.js?v=...	200	http/1.1	script	_ListCache.js'
_listCacheGet.js?v=6a...	200	http/1.1	script	_ListCache.js'
_listCacheHas.js?v=6a...	200	http/1.1	script	_ListCache.js'
_listCacheSet.js?v=6a...	200	http/1.1	script	_ListCache.js'
_Uint8Array.js?v=6ab5...	200	http/1.1	script	_cloneArrayBu
_hashClear.js?v=6ab5...	200	http/1.1	script	_Hash.js?v=6a
_hashDelete.js?v=6ab...	200	http/1.1	script	_Hash.js?v=6a
_hashGet.js?v=6ab586...	200	http/1.1	script	_Hash.js?v=6a
_hashHas.js?v=6ab58...	200	http/1.1	script	_Hash.js?v=6a
_hashSet.js?v=6ab586...	200	http/1.1	script	_Hash.js?v=6a
_isKeyable.js?v=6ab58...	200	http/1.1	script	_getMapData.
_assocIndexOf.js?v=6a...	200	http/1.1	script	_listCacheDel
_nativeCreate.js?v=6a...	200	http/1.1	script	_hashClear.js'
detector-exec.js	200	chrome-exte...	script	detector.js:1
vite.svg	200	http/1.1	svg+xml	Other
141.327ce5c7.js	200	chrome-exte...	script	runtime.fbeea
653 requests 4.9 MB transferred 4.8 MB resources Finish: 4.76 s DOMContentLoaded: 4.49 s Load: 4.71 s				

可以看到，此时发起了600多个请求，这是因为 `lodash-es` 有超过 600 个内置模块！

`vite`通过将 `lodash-es` 预构建成单个模块，只需要发起一个HTTP请求！可以很大程度地提高加载性能

7.3 基本原理

跟着 `debug` 来一步一步看 `vite` 本地是如何工作的

首先从 `package.json` 出发，找到项目启动命令：

```

6   "scripts": {
7     "dev": "vite",
8     "build": "vue-tsc && vite build",
9     "preview": "vite preview"
10  },

```

可以看到，`dev` 对应的命令直接就是 `vite`，然后我们再到 `node_modules` 下面的 `vite` 下面的 `bin` 文件夹下面的 `vite.js` 文件，这就是 `vite` 运行的入口文件。

这里有一个 `start` 方法，从这打上断点开始慢慢往下走，就能够知道整个运行的基本原理

```

43  function start() {
44    return import('../dist/node/cli.js')
45  }

```

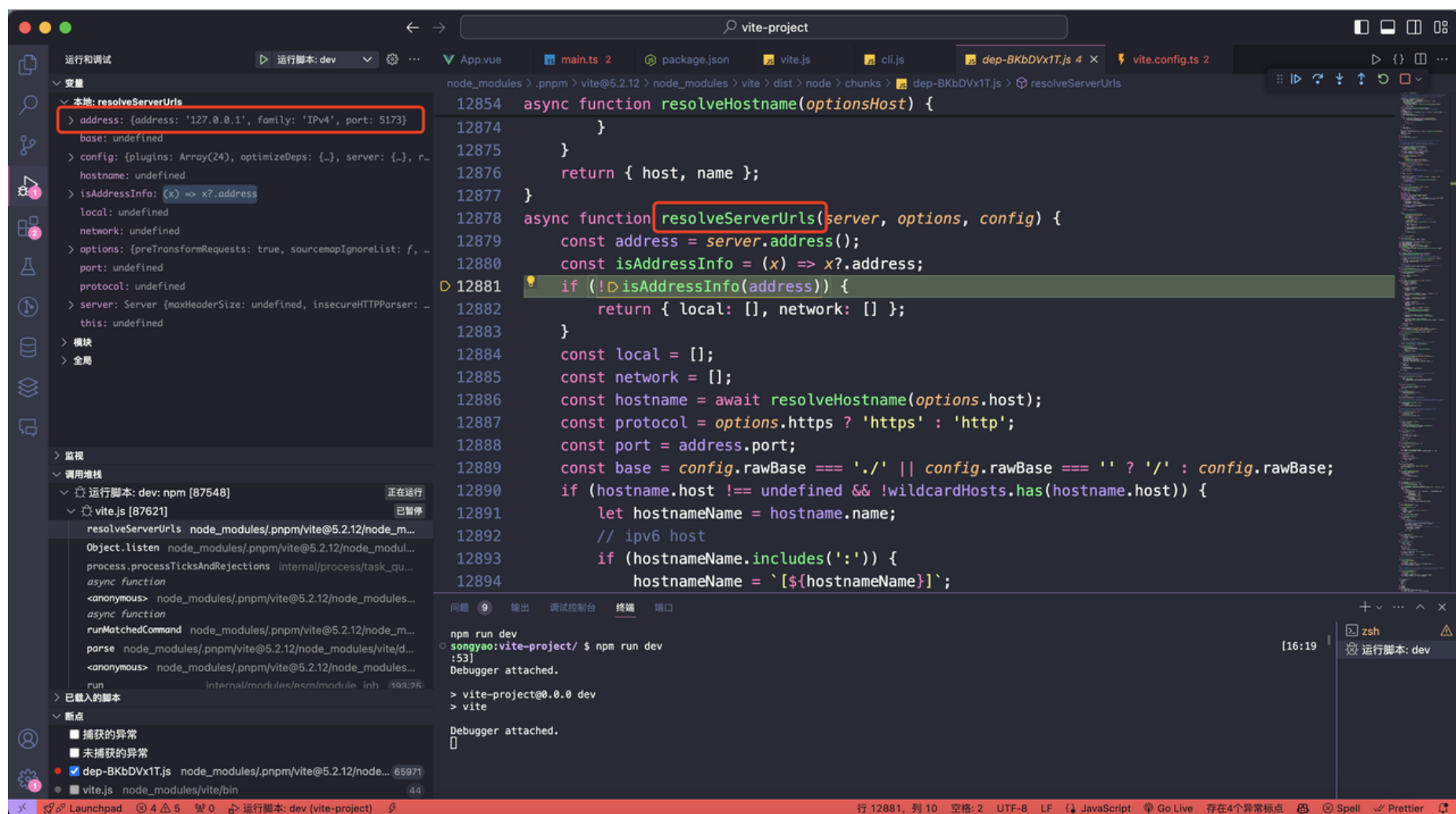
从上面我们知道，`vite` 首先是会启动一个本地服务，基于该服务对文件的请求进行处理返回

```

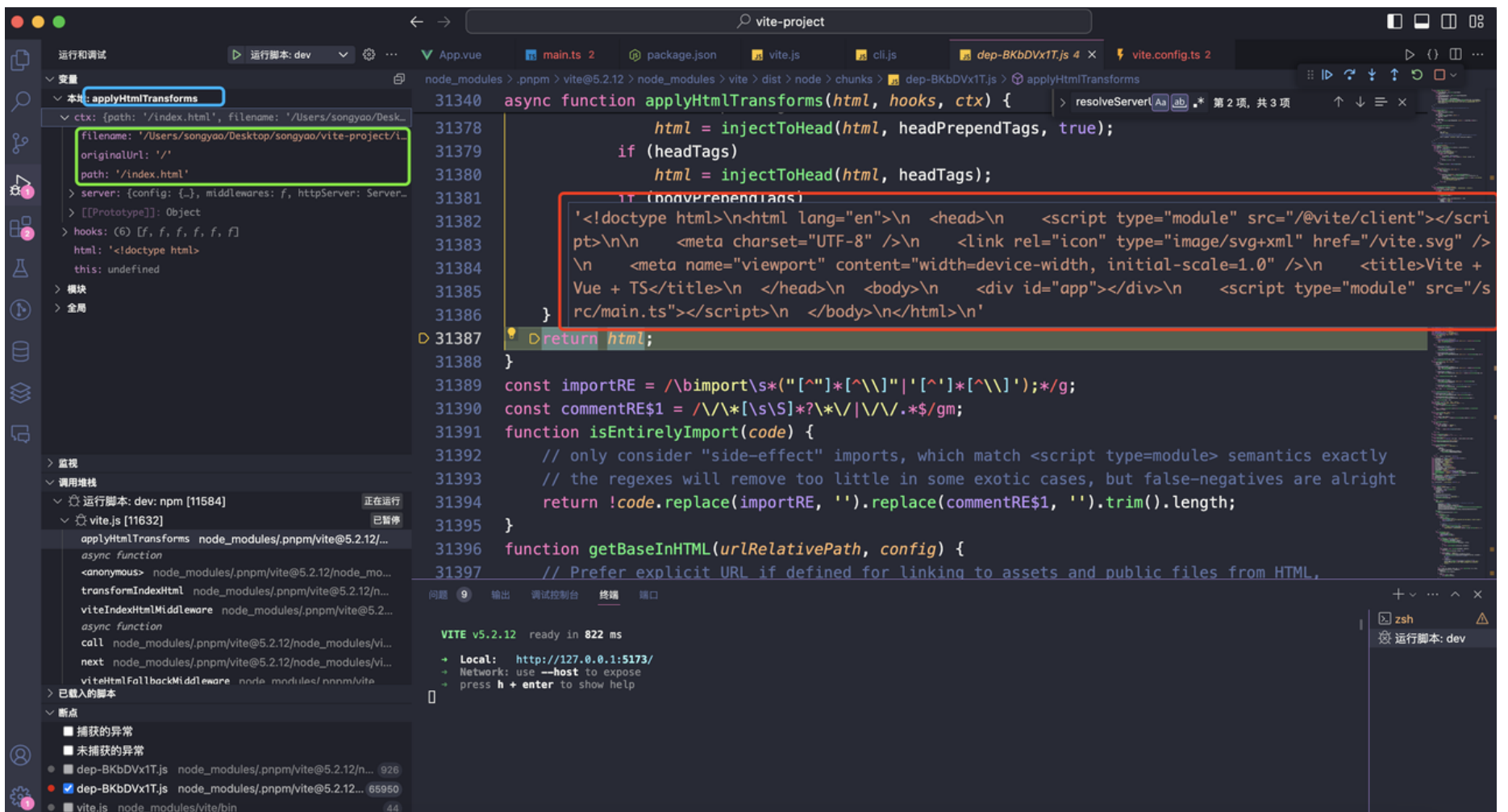
762     const server = await createServer({
763       root,
764       base: options.base,
765       mode: options.mode,
766       configFile: options.config,
767       logLevel: options.logLevel,
768       clearScreen: options.clearScreen,
769       optimizeDeps: { force: options.force },
770       server: cleanOptions(options),
771     });

```

接着往下走，我们可以看到有一个处理url的方法，此时运行栈里面的 `address` 变量也能够看到是 `127.0.0.1:5173`，这就是我们等会要访问的本地服务，当然现在浏览器还什么也看不到，因为还没开始处理 `/` 路由，该路由需要返回一个html文件，也就是我们的模版文件（项目基于Vue3）



继续往下走，就可以看到有一个 `applyHtmlTransforms` 方法用来处理 `html` 文件并返回，可以看到当前请求的原始路径是 `/`，返回的文件是项目根目录下的 `index.html` 文件



里面有一个脚本文件 `<script type="module" src="/src/main.ts"></script>`，接下来就该请求并处理入口文件 `main.ts` 了

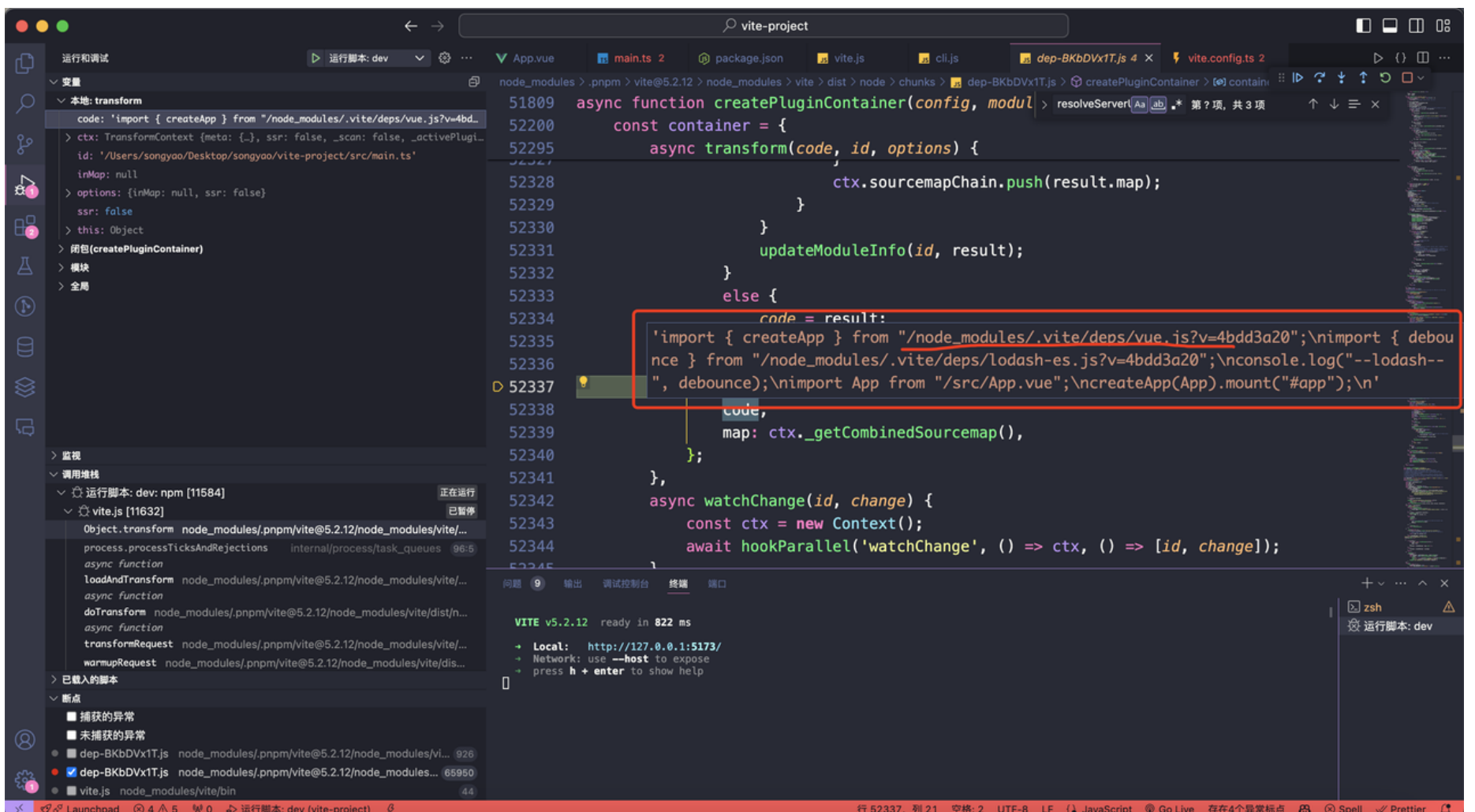
`main.ts` 文件如下：

Code block

```

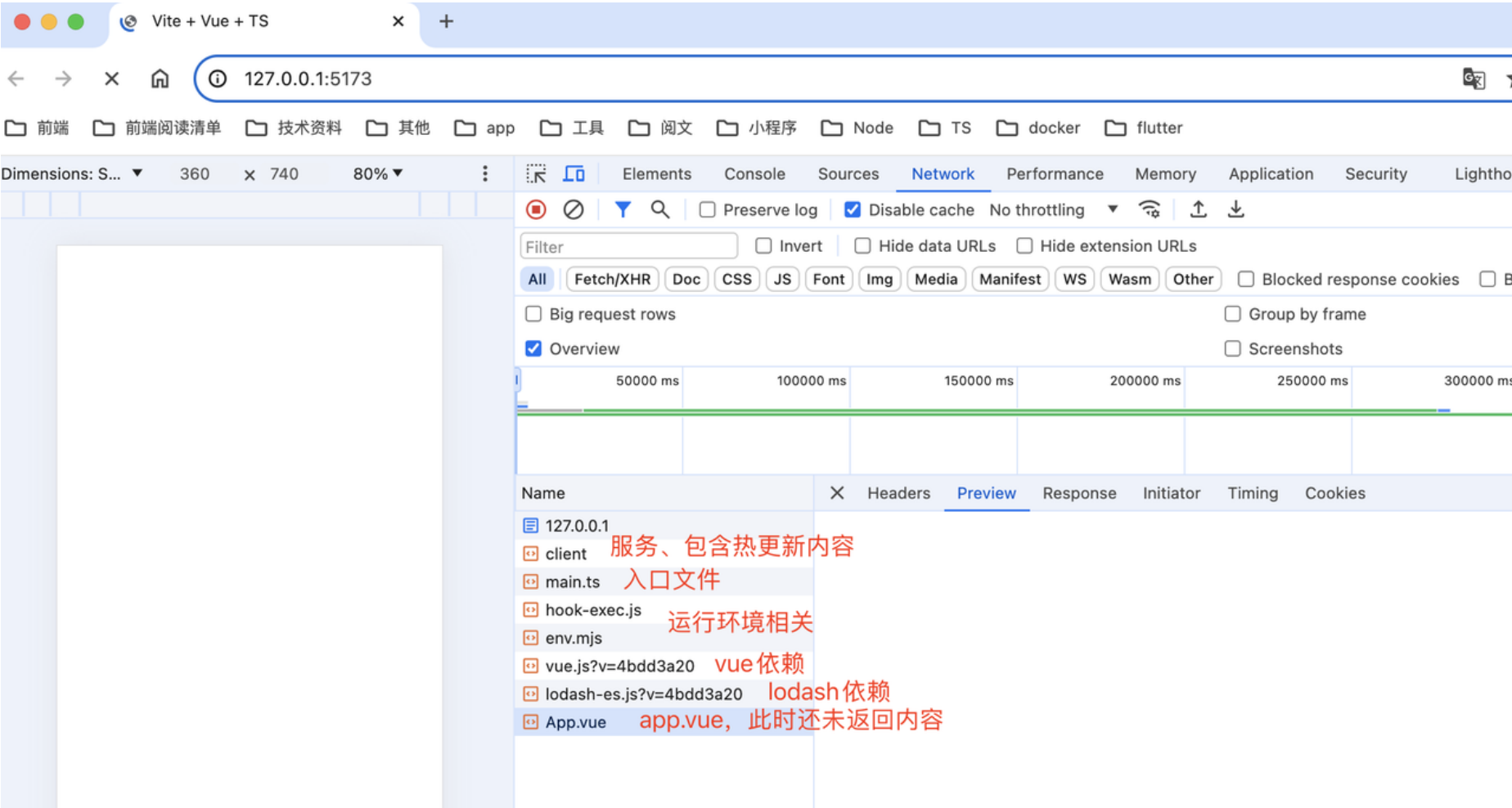
1 import { createApp } from 'vue'// import './style.css'import { debounce } from 'lodash-es'console.log('--lodash--', debounce)
2 import App from './App.vue'createApp(App).mount('#app')
```

经过处理之后变成了：

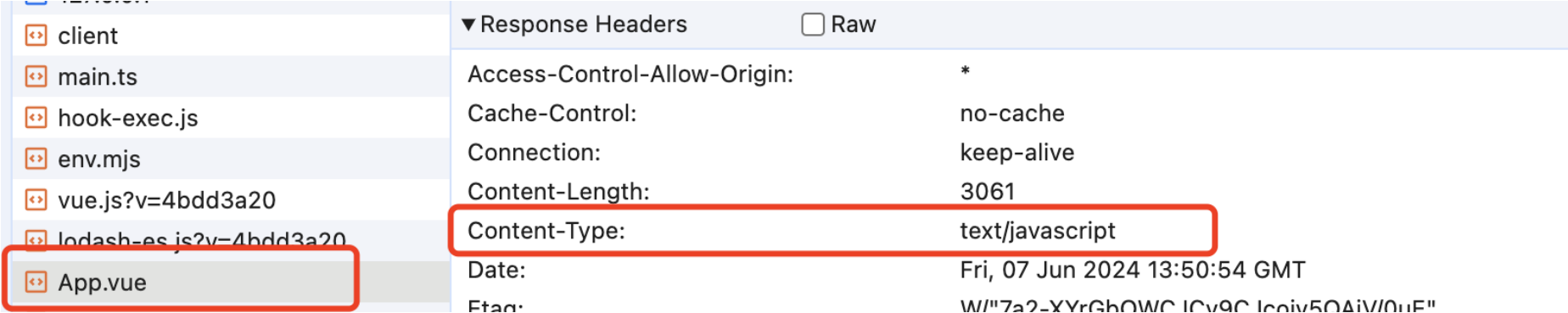


它其实也没做啥处理，只是把依赖的引用路径处理成了预构建下的路径（.vite/deps/），把源码的引用路径处理成了绝对路径。

来看看此时浏览器中的加载顺序：



整个文件的加载顺序是不是都对上了，注意看这个 App.vue 文件，虽然是 .vue 结尾，但文件类型依然是一个 JavaScript 文件



App.vue 经过编译后文件类型已经转成JS了！

编译后：

127.0.0.1

client

main.ts

hook-exec.js

env.mjs

vue.js?v=4bdd3a20

lodash-es.js?v=4bdd3a20

App.vue

App.vue?vue&type=style&index=...

__x00__plugin-vue:export-helper

Preview

Response

Initiator

Timing

Cookies

```
1 import { createHotContext as __vite__createHotContext } from "@vite/client";import.meta.hot = __vite__createHotContext()
2 import { ref } from "node_modules/.vite/deps/vue.js?v=4bdd3a20";
3 const _sfc_main = /*#__PURE__*/_defineComponent({
4   __name: "App",
5   setup(__props, { expose: __expose }) {
6     __expose();
7     const userName = ref("\u524D\u7A7A\u5337\u7396");
8     const __returned__ = { userName };
9     Object.defineProperty(__returned__, "__isScriptSetup", { enumerable: false, value: true });
10    return __returned__;
11  }
12});
13 import { toDisplayString as _toDisplayString, openBlock as _openBlock, createElementBlock as _createElementBlock,
14 const _withScopeId = (n) => (_pushScopeId("data-v-7a7a37b1"), n = n(), _popScopeId(), n);
15 const hoisted_1 = { class: "user name" };
16 function _sfc_render(_ctx, _cache, $props, $setup, $data, $options) {
17   return _openBlock(), _createElementBlock(
18     "div",
19     hoisted_1,
20     _toDisplayString($setup.userName),
21     1
22   );
23 }
24
25 import "/src/App.vue?vue&type=style&index=0&scoped=7a7a37b1&lang.css";
26 _sfc_main.__hmrId = "/a/a37b1";
27 typeof __VUE_HMR_RUNTIME__ !== "undefined" && __VUE_HMR_RUNTIME__.createRecord(_sfc_main.__hmrId, _sfc_main);
28 import.meta.hot.accept((mod) => {
29   if (!mod)
```

所以紧接着又会去请求这个css内容

SFC 组合式 API 被编译成了 _defineComponent

模版内容被编译成了 render 渲染函数

style 部分被编译成了一个 JS 文件

再接着往下走，看下style被编译成了什么内容：

127.0.0.1

client

main.ts

hook-exec.js

env.mjs

vue.js?v=4bdd3a20

lodash-es.js?v=4bdd3a20

App.vue

App.vue?vue&type=style&index=0&scoped...

Preview

Response

Initiator

Timing

Cookies

```
1 import {createHotContext as __vite__createHotContext} from "@vite/client";
2 import.meta.hot = __vite__createHotContext("/src/App.vue?vue&type=style&index=0&scoped=7a7a37b1&lang.css");
3 import {updateStyle as vite_updateStyle, removeStyle as vite_removeStyle} from "@vite/client";
4 const __vite_id = "/Users/songyao/Desktop/songyao/vite-project/src/App.vue?vue&type=style&index=0&scoped=7a7a37b1&lang.css";
5 const __vite_css = "\n.logol[data-v-7a7a37b1] {\n  height: 6em;\n  padding: 1.5em;\n  will-change: filter;\n}\n";
6 vite_updateStyle(__vite_id, __vite_css);
7 import.meta.hot.accept()
8 import.meta.hot.prune(()=>__vite__removeStyle(__vite_id))
```

scopeid, 用于样式隔离

css 内容

最后整个页面就可以渲染出来了

Vite + Vue + TS

127.0.0.1:5173

完成更新

前端

前端阅读清单

技术资料

其他

app

工具

阅文

小程序

Node

TS

docker

flutter

Dimensions: 360 x 740 80%

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

Filter

Preserve log

Disable cache

No throttling

Blocked response cookies

Blocked requests

3rd-party requests

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

WS

Wasm

Other

Big request rows

Group by frame

Screenshots

Overview

50 ms

100 ms

150 ms

200 ms

250 ms

300 ms

350 ms

400 ms

450 ms

500 ms

550 ms

600 ms

650 ms

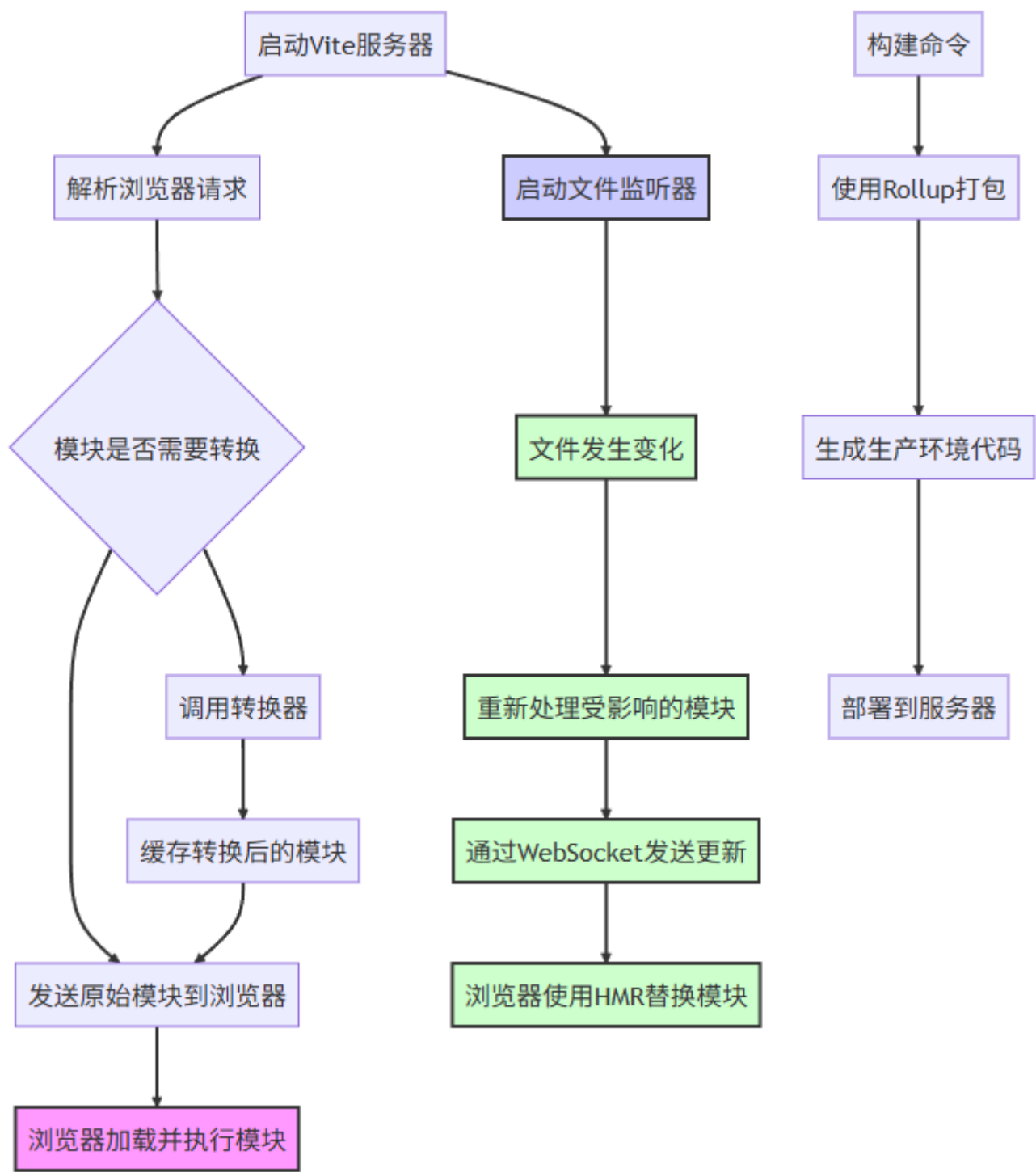
Name	Status	Protocol	Type	Initiator	Size	Time
127.0.0.1	200	http/1.1	document	Other	684 B	53 ms
hook-exec.js	200	chrome-extern...	script	hook.js:1	6.9 kB	38 ms
client	200	http/1.1	script	(index):3	106 kB	72 ms
main.ts?t=17177693984...	200	http/1.1	script	(index):12	1.1 kB	81 ms
env.mjs	200	http/1.1	script	client:1	3.6 kB	82 ms
vue.js?v=4bdd3a20	200	http/1.1	script	main.ts:1	338 kB	54 ms
lodash-es.js?v=4bdd3a20	200	http/1.1	script	main.ts:3	307 kB	65 ms
App.vue?t=1717769398...	200	http/1.1	script	main.ts:6	3.2 kB	74 ms
127.0.0.1	101	websocket	websocket	client.ts:77	0 B	Pending
App.vue?vue&type=style...	200	http/1.1	script	App.vue:7	1.1 kB	19 ms
__x00__plugin-vue:expo...	200	http/1.1	script	App.vue:7	1.4 kB	19 ms
detector-exec.js	200	chrome-extern...	script	detector.js:1	1.2 kB	14 ms
vite.svg	200	http/1.1	svg+xml	Other	1.8 kB	39 ms
141.327ce5c7.js	200	chrome-extern...	script	runtime.fbeaff4.js:1	1.2 kB	4 ms

7.4 总结

`vite` 整体思路：启动一个 `connect` 服务器拦截由浏览器请求 ESM 的请求。通过请求的路径找到目录下对应的文件做一下编译最终以 ESM 的格式返回给浏览器。

对于 `node_modules` 下面的依赖，`vite` 会使用 `esbuild` 进行预构建，主要是为了兼容 CommonJS 与 UMD，以及提高性能。

这样完整走一遍，是不是对 `Vite` 的理解又更深一步了，它实际上就是“走一步看一步”，不像 `webpack` 上来就扫描整个项目进行打包编译，所以 `vite` 的构建速度会比较快



8. Vite 与后续前端生态的发展

<https://voidzero.dev/posts/announcing-rolldown-vite>

8.1 Vite 正在全面 Rust 化

很多人看到这次更新，可能会说：“**Rolldown 不就是个性能更好的打包器吗？用不用都行吧？**”

说实话，这种理解可能有些过于表面了。

这次更新的不仅仅是一个工具，而是把整个的 vite 底层都重写了一遍：

- Vite 的打包器，从 JS 写的 Rollup，换成了 Rust 写的 Rolldown
- 配套的 Babel、Terser、ESLint，也被 Rust 实现的 Oxc 接管
- 整个构建链路，从解析到压缩，从转换到分析，全都 Rust

为什么要这么干呢？

很简单，因为：**JS 写的构建工具已经摸到天花板了。**

不管你怎么做缓存、怎么压缩 AST、怎么优化 Plugin 顺序，JS 就是做不到 Rust 那种级别的执行效率。

而现代前端的项目体积正在变得越来越大，早就不是之前只写几个静态页面的事情了！

目前 微前端、组件库、国际化、权限系统……每加一个功能，构建时间就会变得越来越长，特别是很多公司在配合 `CI/CD` 的逻辑，每构建跑一次可能就得跑 2 分钟，而如果换成 Rolldown 那么就只要 15 秒上下了，你算算整个团队每天省下多少时间？

因此，这样的替换 “势在必行”，同时这也标记着：**Vite 已经不再是一个 JS 写的现代前端工具了，而是一个由 Rust 驱动的、高性能构建内核。**

8.2 真实表现到底快了多少？

rolldown-vite 仍处于开发阶段，但从小项目到大型企业应用程序的早期采用者已经看到了显著的成果。

一些亮点：

- *GitLab* 将构建时间从 2.5 分钟缩短至 40 秒，并将内存使用量减少了 100 倍。
- *Excalidraw* 的构建时间从 22.9 秒缩短至 1.4 秒（**速度提高了 16 倍**）。
- *PLAID Inc.* 发现一个前端的构建时间从 1 分 20 秒缩短至 5 秒（**速度提高了 16 倍**）。
- *Appwrite* 构建时间从 12 分钟以上缩短至仅 3 分钟，内存使用量减少了 4 倍。
- 与 Vite 相比，*Particl* 实现了近 **10 倍的加速**（与 **Next.js** 相比，实现了近 **29 倍的加速**），构建时间从一分钟多缩短到仅 6 秒。

这些结果不仅表明构建速度显著提升，而且在某些情况下，内存占用量也显著降低。如需了解更多详情或分享您自己的结果，请访问 `vitejs/rolldown-vite-perf-wins` 代码库。

哦，有趣的是 - 您现在正在阅读的这篇博文是使用在 Rolldown-Vite 上运行的 VitePress 构建的，并且在 Netlify 上生产构建仅需 1.8 秒。

掘金技术社区 @ 程序员Sunday

8.3 尤雨溪在下一盘大棋

很多同学可能会说：**Vite 已经“遥遥领先”了，为啥还非要换底层呢？多麻烦呀！**

如果你有这种疑惑的话，那么可能是因为你对 `vite` 使用到的这些新工具还不太了解，如果你了解一下背后的发布方，就知道这件事没那么简单。

Rolldown 是谁发布的？不是 Vue，也不是 Vite 核心团队，而是尤雨溪创办的新公司 —— **VoidZero (也叫做 void(0))**。想要详细了解的，可以看下我之前发的这篇文章 [尤雨溪新公司 Void\(0\) 首款产品发布，竟然是它...](#)

这是一家由 尤雨溪 创建的专门做 JavaScript 工具链的开源公司。关于这一块的介绍，可以看这篇博客 [尤雨溪创建 VoidZero，并得到 460 万美金融资](#)

这家公司刚一出手就连放两个大招：

- 第一个是 **Oxc**：这是一个全新的 Rust 实现 JS 工具链（parser、transform、minifier、linter、formatter，全都自己造）
- 第二个就是 **Rolldown**：Vite 打包器的 Rust 替代方案，目标直接瞄准 Rollup、Babel、Terser 这条传统链路

而这次 Vite 接入 Rolldown，正是 void(0) 把自家工具「回注入」开源生态的第一步。

所以这不是在“优化 Vite”，而是想要 “**替换整条构建基础设施**”

你可以这么理解 void(0) 的策略路径：

1. Vue 站稳前端框架圈核心位置
2. Vite 用 Rollup 起家，成为构建工具主流选择
3. void(0) 作为新公司登场，切入工具链底层，用 Rust 重写一整套生态
4. 再反哺 Vite，用 Rolldown 替代原来的 JS 构建方案
5. 最终形成：**Vue + Vite + void(0) 工具链** 的闭环

这其实是一个很聪明、很清晰的长期路线图：**不再被 Babel、Terser、ESLint 等“生态外依赖”所绑定，而是自己控制工具底层、性能节奏、开发体验。**

尤雨溪本人也在社区里反复提过：Vite 的未来，不只是“构建工具”，而是下一代工程化的“**前端开发基建平台**”。

而这张底牌，就是 Rolldown + Oxc。

你可以想想看，如果：

- Vue 生态已经在试水 Rolldown
- Vite 即将全面接入 Rolldown
- Vite 插件作者必须适配 Rolldown（否则未来会不兼容）

那就意味着：

无论你是 Vue、React、Svelte，还是用 Vite 的任何框架，都必须配合这次“**Rust 工具链**”的迁移。否则将有可能会被踢出前端生态。